



Universidad Nacional Experimental de Guayana

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

LOS LENGUAJES DE PROGRAMACIÓN

Profesor:

Márquez, Félix

Autores:

Longart, Daniela V-31445710

Martínez, Fabiana V-30498516

Ortiz, Sebastián V- 30576350

Valencia, Haddan V-31818222

Ciudad Guayana, 03 de junio del 2026.

Resumen Académico

El propósito de este estudio es analizar de forma analítica y profunda la infraestructura computacional de los lenguajes de programación mediante un enfoque comparativo y empírico. Se evaluaron cuatro entornos tecnológicos diferenciados: Zig, Rust, Python y JavaScript, examinando sus fronteras morfológicas, estructuras sintácticas y mecanismos de ejecución. Para abordar el rendimiento en entornos con procesamiento intensivo, se implementó un benchmark controlado basado en el cálculo de la ecuación de segundo grado para vectores de tamaño $n=200$ sobre un procesador Intel Core i3-4005U. Los resultados empíricos demostraron disparidades significativas: Python registró un tiempo promedio de 0.6504 ms (15 MB de memoria), seguido por Rust con 85.5797 ms (<1.5 MB), JavaScript con 248.56 ms (30 MB) y Zig con 336.97 ms (<1.0 MB). Mientras los lenguajes compilados nativamente (LLVM) priorizaron un aislamiento estricto y consumo mínimo de memoria pico, el motor dinámico interpretado evidenció un comportamiento optimizado para la carga vectorial lineal. Como complemento de abstracción, se diseñó el Lenguaje L, un Lenguaje de Dominio Específico (DSL) con tipado estático y sintaxis autocontenida en español. Su gramática fue formalizada mediante la notación de Backus-Naur (BNF) para el control del sistema crítico de microrredes eléctricas inteligentes ECO-GRID. La innovación sintáctica del DSL se validó mediante el desarrollo riguroso de dos scripts operativos enfocados en la prevención de fugas térmicas en baterías y el balance autónomo de carga energética. El estudio concluye que las decisiones de diseño léxico y gramatical impactan directamente en la expresividad y eficiencia del software industrial.

Introducción

En los ciclos iniciales de la ingeniería, los estudiantes interactúan con los lenguajes de programación únicamente como herramientas estáticas para traducir lógica algorítmica elemental. Sin embargo, la ciencia de la computación exige trascender esta perspectiva superficial para concebir los lenguajes como productos arquitectónicos minuciosamente diseñados bajo restricciones formales rigurosas.

El planteamiento fundamental de este problema radica en comprender cómo las fases iniciales del compilador e intérprete procesan el código fuente. A nivel morfológico y léxico, el analizador escanea caracteres crudos para estructurar secuencias de tokens válidos (palabras reservadas, identificadores y literales), lidiando con discrepancias críticas como el descarte de elementos irrelevantes frente a la indentación con valor semántico. A nivel sintáctico, el parser evalúa dicha cadena de tokens bajo gramáticas libres de contexto para construir el Árbol de Sintaxis Abstracta (AST), determinando la jerarquía de las estructuras de control y la precedencia de operadores.

En la industria contemporánea del software, analizar la eficiencia estructural y la convergencia multiparadigma no es un ejercicio teórico, sino una necesidad operativa frente a demandas de mercado cambiantes. Esta investigación justifica la necesidad de evaluar empíricamente el impacto de los mecanismos de ejecución (compilación nativa frente a interpretación y JIT) en entornos de procesamiento intensivo. Asimismo, introduce la relevancia de los Lenguajes de Dominio Específico (DSL) como soluciones óptimas para proporcionar abstracción y seguridad en Sistemas Lógico-Físicos Críticos, aislando la complejidad del hardware en favor de interfaces Hombre-Máquina legibles y confiables.

Desarrollo

Actividad I. Matriz de Paradigmas

Actualmente en la ingeniería de software ya no se requiere la pureza paradigmática, se ha reemplazado casi en su totalidad por una ideología pragmática que indica que, en vez de buscar verdades absolutas, se debe sostener un pensamiento que debe ser útil para resolver problemas y adaptarse al entorno. A raíz de esto, nació la convergencia multiparadigma.

Este es un fenómeno donde los lenguajes de amplio propósito adoptan características funcionales u orientadas a objetos simultáneamente para responder las demandas del mercado. Por ejemplo, lenguajes como Python, JavaScript o Rust ilustran cómo las restricciones de un solo paradigma limitan la expresividad. En la industria se requiere robustez de encapsulamiento, que es propio de la orientación a objetos, pero también la inmutabilidad y el manejo de funciones de orden superior, propios del paradigma funcional.

Este cambio no es accidental, responde a una decisión de diseño y léxico que impacta de manera directa en la eficiencia, la expresividad y la mantenibilidad del software. A continuación, se realizará una matriz descriptiva de los paradigmas clásicos y de los emergentes:

Tabla 1

Matriz de paradigmas

Paradigma	Descripción	Ejes temáticos	Mecanismo de implementación	Manifestación en lenguajes modernos
Imperativo / Estructural	Basado en una secuencia de instrucciones que modifican el estado del programa.	- Gestión explícita del estado: El programador controla paso a paso cómo cambia la memoria. - Secuenciación:	Se basa en la modificación directa del estado del programa mediante instrucciones secuenciales que alteran	C, Pascal, Zig (en su modo más básico).

Orientado a Objetos (POO)	Organiza el software mediante objetos que combinan datos y comportamiento	Ejecución estricta de instrucciones. - Mutabilidad y Efectos secundarios: Las variables cambian de valor en el tiempo, lo que genera efectos secundarios en la memoria global del sistema. -	posiciones de memoria.	
		Encapsulamiento y Polimorfismo: Ocultamiento del estado interno y capacidad de una interfaz para representar múltiples formas subyacentes. - Herencia vs. Composición: Mecanismos de reutilización de código. - Abstracción: Basada en la combinación de datos y comportamiento dentro de entidades llamadas objetos. - Inmutabilidad y Funciones puras: Los datos no mutan y las funciones son tratadas como ciudadanos de primer orden.	El polimorfismo dinámico se implementa comúnmente a través de tablas de métodos virtuales (v-tables), lo que añade una penalización menor en tiempo de ejecución debido a la resolución indirecta de funciones. Java, C#, Python, Ruby. La POO modela el mundo como objetos que contienen estado y métodos.	
Funcional	Basado en funciones matemáticas y datos inmutables.	- Evaluación perezosa: Los valores se calculan solo cuando son estrictamente	Las funciones de primer orden requieren que el compilador gestione cierres (closures), reservando memoria en el heap para variables capturadas.	Haskell, Elixir, F#, e incorporado en Rust (iteradores), JavaScript (map/filter/reduce) y Python (funciones lambda).

		necesarios.		
		- Transparencia referencial:		
		Eliminación programática de efectos colaterales, garantizando que una función siempre retorne el mismo resultado con los mismos argumentos.		
		- Programación basada en relaciones:	El motor de ejecución (frecuentemente basado en la Máquina Abstracta de Warren) busca resolver metas mediante un proceso de unificación (un emparejamiento de patrones bidireccional) y backtracking (búsqueda en árbol con retroceso) cuando una ruta falla.	
Lógico / Declarativo	El programador define relaciones y reglas en lugar de pasos de ejecución.	- Unificación y Cláusulas de Horn: Mecanismos matemáticos para la resolución de inferencias lógicas.		Prolog, Datalog. El programador declara hechos y reglas; el motor de inferencia resuelve las consultas.
		- Abstracción de flujo: Abstracción total del flujo de control por parte del programador, delegándolo al motor de inferencia.		
Concurrente / Actores	Se centra en la ejecución simultánea de múltiples procesos independientes.	- Paso de mensajes: Comunicación asíncrona entre entidades autónomas.	En el modelo de actores (ej. Erlang/Elixir) los procesos no comparten memoria. Cada actor posee un buzón (mailbox) aislado y secuencial.	Erlang, Elixir, Akka (para Scala/Java). Cada actor es una entidad ligera con su propio estado y cola de mensajes.
		- Aislamiento de estado: Aislamiento estricto de estado entre entidades concurrentes.		

- Mitigación de errores:
Mitigación de condiciones de carrera directamente a nivel de diseño lingüístico.

Actividad II. Código fuente analizado y tabla de benchmarking

Para el desarrollo de esta actividad, tomamos el ejemplo 2: Cálculo de ecuación de segundo grado para vectores a,b,c con cantidad de elemento n para n=200.

Análisis morfológico (léxico) de Zig, Rust, Python y JavaScript

El nivel morfológico, o análisis léxico, constituye la fase inicial en el procesamiento de un lenguaje de programación. En esta etapa, el analizador léxico escanea el código fuente carácter por carácter para agruparlos en unidades con significado lógico denominadas *tokens* (identificadores, palabras clave, literales y operadores). A continuación, se detalla el comportamiento léxico de los cuatro lenguajes evaluados:

1. Gestión de palabras reservadas e identificadores: Los cuatro lenguajes poseen conjuntos finitos de palabras reservadas, pero difieren sustancialmente en la rigurosidad de su declaración:
 - a. Zig y Rust (Lenguajes de Sistemas): Exigen una declaración explícita e inmutable de la memoria. Zig emplea el token `const` para referencias inmutables y `var` para variables mutables. Rust, por defecto, asume la inmutabilidad con `let`, requiriendo explícitamente el token compuesto `let mut` para permitir mutabilidad.

- b. JavaScript (JIT / Multiparadigma): Adopta un enfoque similar al usar `const` y `let` para delimitar el alcance de las variables (scope de bloque), abandonando en la práctica el uso histórico del token `var`.
 - c. Python (Interpretado / Dinámico): Presenta la mayor abstracción morfológica. Prescinde de tokens de declaración como `let` o `const`; la asignación de un literal a un identificador (ej. `n = 200`) es suficiente para inferir el tipo y reservar la memoria dinámicamente.
2. Delimitadores y Estructura de Bloques (Tratamiento de Elementos Irrelevantes): La discrepancia morfológica más notable reside en la definición de las fronteras de los bloques lógicos y el manejo de los caracteres en blanco:
- a. El modelo C-like (Zig, Rust, JavaScript): Estos lenguajes ignoran los espacios en blanco, tabulaciones y saltos de línea (considerados tokens irrelevantes). Para estructurar el alcance de funciones y bucles, emplean delimitadores explícitos: llaves de apertura `{` y cierre `}`. Además, utilizan el punto y coma `;` como token terminal obligatorio para indicar el fin de una sentencia lógica.
 - b. El modelo de indentación significativa (Python): Python elimina los tokens de delimitación clásicos (`{`, `}`, `;`). En su lugar, el analizador léxico otorga valor semántico a los espacios en blanco. Un bloque lógico se define mediante los tokens de salto de línea y el nivel exacto de indentación (típicamente 4 espacios) precedidos por el token de dos puntos `:`. Esto fuerza una legibilidad visual estandarizada desde el diseño mismo del lenguaje.
3. Tipado explícito contra el dinámico: La forma en que el lexer (analizador léxico) interpreta los valores crudos (literales) difiere según el paradigma de seguridad:

- a. Rust y Zig: Exigen que los literales numéricos o las estructuras incluyan declaraciones de tipo cuando el compilador no puede inferirlo con total seguridad. En Rust se observa en tokens como `Vec<f64>` para forzar un vector de punto flotante de 64 bits. En Zig, se refleja en los parámetros del asignador de memoria (`allocator.alloc(f64, n)`).
- b. Python y JavaScript: Como lenguajes de tipado dinámico, el analizador léxico procesa literales como `[-5.0]` (Python) o `[-5.0]` (JS) directamente como colecciones flexibles, donde el motor subyacente (CPython o V8) asigna los tipos de datos en tiempo de ejecución.

Análisis Sintáctico: Jerarquía y Estructuras de Control

En la fase de análisis sintáctico, el *parser* evalúa la secuencia de tokens generada por el analizador léxico para construir un Árbol de Sintaxis Abstracta (AST) . Esta estructura jerárquica valida que las sentencias respeten la gramática libre de contexto del lenguaje. A continuación, se documenta la sintaxis de las estructuras esenciales empleadas en los algoritmos de prueba:

1. Definición de Subprogramas (La función principal).

El punto de entrada del programa revela cómo cada gramática estructura la firma de sus subrutinas:

- Rust (`fn main() {}`) y Python (*flujo principal implícito o explícito con `def`*): Poseen una sintaxis directa donde el nodo raíz de la función contiene un bloque de código estándar. El tipo de retorno está implícito o se asume estático.
- Zig (`pub fn main() !void {}`): Su árbol sintáctico es más complejo. El analizador exige modificadores de visibilidad (`pub`) y, de manera particular, incorpora el manejo de errores directamente en la sintaxis de retorno (`!void`). El símbolo `!` obliga al analizador

sintáctico a envolver la función en una estructura capaz de devolver un error o "nada" (void), demostrando un diseño de sistemas altamente seguro.

2. Jerarquía Sintáctica de las Estructuras Iterativas (Bucles)

El procesamiento intensivo del algoritmo cuadrático se logra mediante estructuras repetitivas, cuyas jerarquías difieren radicalmente entre los lenguajes evaluados:

El enfoque Iterador (Rust y Python):

- Rust: `for i in 0..n { ... }`
- Python: `for i in range(n): ...`

En ambos casos, el árbol sintáctico no construye un contador manual. El nodo del bucle (ForLoop) evalúa una expresión iteradora (`0..n` o `range(n)`). El analizador abstrae la inicialización, la condición de parada y el incremento, delegando esto a un objeto iterable. Esto genera un AST más limpio y propenso a optimizaciones automáticas por parte del compilador.

El enfoque C-Estándar (JavaScript):

- `for (let i = 0; i < n; i++) { ... }`

La jerarquía exige exactamente tres nodos hijos dentro de la declaración del bucle: un nodo de inicialización, un nodo de evaluación booleana y un nodo de mutación (incremento).

El enfoque de Continuación (Zig):

- `while (i < n) : (i += 1) { ... }`

Zig prescinde del clásico bucle for numérico. Su analizador sintáctico procesa un `while` que recibe una expresión booleana principal (`i < n`) y, sintácticamente, se le adjunta una "expresión de continuación" delimitada por dos puntos `: (i += 1)`. El AST vincula esta última expresión para que se ejecute obligatoriamente al final de cada iteración del bloque.

3. Precedencia y Evaluación de Expresiones Matemáticas

Dentro del bucle, los compiladores deben jerarquizar el cálculo del discriminante y las raíces:

Figura 1

Función cuadrática

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Sintácticamente, los cuatro analizadores (tanto compiladores de Zig/Rust como motores de JS/Python) aplican reglas formales de precedencia de operadores. Por ejemplo, en la expresión $b[i] * b[i] - 4.0 * a[i] * c[i]$:

1. Los accesos a los arreglos (ArrayAccess) tienen la prioridad más alta en el árbol.
2. Las multiplicaciones (*) se agrupan como sub-árboles inferiores antes de la resta.
3. El nodo raíz de esta expresión matemática es la operación de sustracción (-), que se resuelve sólo cuando sus ramas izquierda y derecha han sido completamente evaluadas.

Tabla benchmarking

Especificaciones exactas de la máquina donde corren las pruebas:

- Procesador (CPU): Intel(R) Core(TM) i3-4005U CPU 1.70GHz, 1700Mhz
- Memoria RAM: 8 GB DDR3
- Sistema Operativo: Windows 10 Home
- Condiciones de ejecución: Sistema en reposo, sin aplicaciones en segundo plano de alto consumo, promediando 5 ejecuciones consecutivas.

Tabla 2*Contraste de compilación entre lenguajes de programación*

Lenguaje de programación	Paradigma Dominante	Mecanismo de Ejecución y Compilación	Tiempo de Ejecución Promedio (ms)	Consumo de Memoria Pico (MB)
		Compilación		
Zig	Imperativo/Estructurado	Nativa (LLVM)	336.97	< 1.0
Python	Multiparadigma (OO, Imperativo)	Interpretado (CPython / VM)	0.6504	15
Rust	Multiparadigma (Funcional, Imperativo)	Compilación Nativa (LLVM)	85.5797	< 1.5
JavaScript	Multiparadigma (Prototípico, Funcional)	JIT (Just-In Time) / V8 Engine	248.56	30

Actividad III. Documentación y programas operativos de lenguaje L***Diseño del Lenguaje L***

Para completar esta serie de actividades, se finaliza con el proceso de diseño de un Lenguaje de Dominio Específico (DSL) creado para la operación y control del sistema de microrredes eléctricas inteligentes con almacenamiento energético ECO-GRID; esta toma el nombre de Lenguaje L. A diferencia de los lenguajes de propósito general analizados en la

Actividad II, los cuales ejecutan comandos generales para que los ordenadores y dispositivos funcionen (Chakray, 2018), un DSL se diferencia al estar construido para un dominio de aplicación específico, restringiendo el alcance global en favor de expresividad, seguridad y facilidad de uso por parte de operadores técnicos (Montenegro, Cueva, Sanjuán y Gaona, 2010).

El área de aplicación elegida, exige que las maniobras de control otorguen alta confiabilidad y respuestas en tiempo real, administradas por el personal de planta que no dependa de formación avanzada en ciencias de la computación. Para cumplir este requisito, el DSL comparte los principios fundamentales que conforman estos tipos de lenguajes de programación.

En primer lugar, las estructuras de control creadas emplean palabras clave en español, buscando simular el lenguaje natural manejado por el operador de planta (a esto se le conoce como sintaxis autocontenida). Por otro lado, se utiliza el tipado estático explícito para la detección de errores de tipo en tiempo de compilación, garantizando la robustez del sistema crítico (López y Montenegro, 2022).

Otro componente importante es la implementación de una semántica clara, ausente de ambigüedades al otorgar una única interpretación posible, dicha acción la complementa el principio de abstracción. Las instrucciones primitivas propias del entorno ECO-GRID son invocadas sin necesidad de conocer los protocolos de comunicación con los actuadores físicos.

Componentes Formales del Lenguaje L

Un elemento que integra al DSL Lenguaje L es un analizador léxico (lexer) que reconoce y clasifica los elementos indivisibles conocidos como tokens (Zea, 2023). La siguiente tabla presenta el inventario completo de tokens válidos en el lenguaje.

Tabla 3

Registro de tokens válidos en el Lenguaje L

Categoría	Descripción	Tokens significativos
Palabras reservadas	Palabras exclusivas del sistema.	si, mientras, entonces, hacer, int, float, string, bool, verdadero, falso
Identificado res	Nombres de las variables y equipos	bat_principal, solar_input, temperatura_actual, excedente
Funciones predefinidas	Acciones directas para controlar la microred.	iniciar_sistema(), medir_temp(), nivel_bateria(), activar_rele(), vender_energia(), aislar_sector(), activar_refrigeracion(),
Literales numéricos	Números enteros y decimales.	55, 0, 2000, 90.5, 55.0
Literales de cadena	Textos para mensajes al lenguaje.	"ALERTA CRITICA: Fuga térmica detectada", "MODO CRITICO"
Literales booleanos	Valores de verdad.	Verdadero, Falso
Operadores relacionales y aritméticos	Símbolos matemáticos y de comparación.	>, <, >=, <=, ==, !=, +, -, *, /, &&, \
Delimitador es	Símbolos para organizar el código.	; (fin de instrucción), { } (bloques), () (parámetros), = (asignación), // (inicio de comentario de línea)

Además de las reglas establecidas anteriormente, es necesaria la aplicación de reglas léxicas complementarias, así el DSL toma un carácter más robusto. En el Lenguaje L es sensible al uso de mayúsculas y minúsculas, por lo que las variables definidas deben escribirse con claridad. El sistema ECO-GRID posee identificadores predefinidos y puede reconocer automáticamente los nombres de los componentes físicos de la planta (bat_principal, solar_input, sector_industrial, panel_solar_array, area_no_critica, etc.) como constantes de hardware; representan la interfaz directa con los sensores y actuadores del sistema.

Otros aspectos secundarios que complementan las reglas léxicas complementarias son:

- Comentarios: Los comentarios inician con los caracteres “//” y se extienden hasta el final de la línea. El analizador léxico los descarta completamente sin generar token alguno.
- Espacios en blanco: Los espacios en blanco (espacios, tabuladores, saltos de línea) son ignorados, excepto cuando actúan como separadores obligatorios entre tokens (ej. entre int y el nombre de una variable).
- Separación de tokens: Todo token debe estar separado de su vecino por al menos un delimitador o carácter de espacio en blanco, salvo los operadores.

Siguiendo los requisitos que pide el sistema ECO-GRID, se muestra a continuación un resumen con los comandos impuestos al Lenguaje L para la interacción con el hardware y la toma de decisiones; las siguientes palabras clave no pueden ser utilizadas con otro propósito que no sea el definido.

Tabla 4

Registro de palabras clave obligatorias

Categoría	Palabra clave	Uso
a		
Inicialización	iniciar_sistema()	Configura la interfaz con los controladores de la microrred.
Sensado	medir_temp(identificador)	Revisa la temperatura (°C) de un banco de baterías.
Sensado	nivel_bateria(identificador)	Retorna el porcentaje de carga (0-100) de un banco de baterías.
Actuación	activar_rele(identificador, comando)	Permite encender y/o apagar los equipos.
Estructura condicional	si (condición) entonces { ... }	Permite tomar una decisión.
Estructura iterativa	mientras (condición) hacer { ... }	Repite una acción mientras la condición se mantenga verdadera.

Adicionalmente, se han incorporado por decisión de diseño los siguientes datos primitivos (datos originales integrados al lenguaje) para gestionar la microrred eléctrica.

Tabla 5

Datos primitivos integrados en el Lenguaje J

Función	Descripción
vender_energia(cantidad)	Exporta energía a la red pública.
aislar_sector(identificador)	Apaga zonas secundarias.

activar_refrigeracion(identificador)	Enciende el enfriamiento.
leer_generacion(identificador)	Mide la energía producida.
leer_demanda(identificador)	Mide la energía consumida.
obtener_hora()	Da la hora actual.
escribir_log(mensaje)	Guarda un registro o nota.
esperar(milisegundos)	Pausa el programa.

Aplicación de Notación BNF

Zea (2023) señala que, en ciertas áreas, los lenguajes de programación generales pueden resultar inadecuados para abordar problemas específicos, como es el caso de las gramáticas, las cuales "pueden ser más fácilmente descritas mediante BNF". En el caso de la gramática del Lenguaje L, este define de manera formal qué secuencias de tokens constituyen programas sintácticamente correctos. A continuación se presenta la especificación utilizando la notación de Backus-Naur (BNF) que establece:

- = significa "se define como".
- | separa alternativas.
- { ... } indica repetición (cero o más veces).
- [...] indica opcionalidad.
- Los símbolos entre “comillas dobles” son terminales (tokens literales).
- Los símbolos sin comillas son no terminales (categorías gramaticales).

Figura 2

Especificación de gramática usada en el Lenguaje L

```

<programa>      ::= { <sentencia> }

<sentencia>      ::= <declaracion> ";"
                  | <asignacion> ";"
                  | <condicional>
                  | <iteracion>
                  | <llamada> ";"

<declaracion>    ::= <tipo> <identificador> [ "=" <expresion> ]

<tipo>           ::= "int" | "float" | "string" | "bool"

<asignacion>     ::= <identificador> "=" <expresion>

<condicional>    ::= "si" "(" <expresion> ")" "entonces" "{" <programa> "}"

<iteracion>      ::= "mientras" "(" <expresion> ")" "hacer" "{" <programa> "}"

<llamada>        ::= <identificador> "(" [ <argumentos> ] ")"

<argumentos>     ::= <expresion> { "," <expresion> }

<expresion>      ::= <literal>
                  | <identificador>
                  | <expresion> <operador> <expresion>
                  | "(" <expresion> ")"

<literal>        ::= <numero> | <booleano> | <cadena>

<numero>         ::= [0-9]+ ( "." [0-9]+ )?

<booleano>       ::= "verdadero" | "falso"

<cadena>         ::= "'" ( [a-zA-Z0-9 \u00E1-\u00FA\u00F1\u00D1] )* "'"

<operador>       ::= ">" | "<" | ">=" | "<=" | "==" | "!="
                  | "+" | "-" | "*" | "/"
                  | "&&" | "||"

<identificador>  ::= [a-z] ( [a-zA-Z0-9_] )*

```

Nota: En el rol del compilador, la notación BNF representa el manual de instrucciones estructurales y un mapa de validación que utiliza el analizador para verificar la construcción del código.

Al momento de escribir alguna instrucción, se deben tomar en cuentas ciertas normas particulares adoptadas para la flexibilidad y el control del comportamiento del Lenguaje L.

Entre ellas se encuentran:

- Soporte de caracteres especiales como los acentos y la letra ñ/Ñ (adaptación en <cadena>).
- Combinación de texto y números usando el operador "+" al realiza una conversión automática del valor numérico a texto.

- La interpretación del código queda a la supervisión del analizador (parser), que otorga prioridad en las operaciones matemáticas (*, /, +, -) y comparaciones lógicas.
- Las operaciones del mismo nivel jerárquico se resuelven estrictamente de izquierda a derecha (asociatividad a la izquierda).
- Se requiere el uso de paréntesis en el código para evitar confusiones en fórmulas complejas.

Aplicaciones en Escenarios Operativos de la Planta ECO-GRID

A continuación se presentan dos programas escritos en Lenguaje L, que resuelven los escenarios operativos exigidos por la especificación ECO-GRID que serán explicados a continuación. Ambos programas cumplen estrictamente con la gramática definida en la sección anterior.

Escenario Operativo A: Prevención de Fuga Térmica y Gestión de Alivio de Carga

El banco de baterías *bat_principal* debe ser monitoreado continuamente. Si su temperatura supera el umbral crítico de seguridad (55°C), el sistema debe activar la refrigeración auxiliar, detener el ingreso de energía proveniente de los paneles solares (para evitar mayor estrés térmico) y desviar el consumo del sector industrial hacia la red comercial de respaldo.

Código en Lenguaje L:

```
// Escenario A: Control de fuga térmica y alivio de carga

// Declaración de variables con tipado estático explícito
float temperatura_actual;
bool alerta_activa;

// Inicialización del sistema de control
iniciar_sistema();
```

```

// Estado inicial de la bandera de alerta
alerta_activa = falso;

// Bucle infinito de monitoreo continuo
mientras (verdadero) hacer {

    // Lectura del sensor térmico del banco de baterías
    temperatura_actual = medir_temp(bat_principal);

    // Evaluación de condición crítica de temperatura
    si (temperatura_actual > 55.0) entonces {

        // Activación del sistema de enfriamiento auxiliar
        activar_refrigeracion(bat_principal);

        // Apertura del relé que conecta la entrada solar (detiene
carga)
        activar_rele(solar_input, "abrir");

        // Conmutación del sector industrial hacia la red comercial
        activar_rele(sector_industrial, "conectar");

        // Registro del evento en el historial del sistema
        escribir_log("ALERTA CRITICA: Fuga térmica detectada en
bat_principal");

        // Actualización de la bandera de alerta (solo una vez por
evento)
        si (alerta_activa == falso) entonces {
            alerta_activa = verdadero;

```

```

    }

    }

    // Pausa de 2 segundos antes de la siguiente lectura
    esperar(2000);

}

```

La secuencia operativa del sistema se inicia con la fase de declaración, donde se reserva espacio para una variable real (*temperatura_actual*) y una variable booleana usada como bandera (*alerta_activa*). De inmediato se procede a la Inicialización, etapa en la que la función *iniciar_sistema()* establece la comunicación con los controladores de la microred y la bandera se fija a *falso* ante la ausencia de una alerta activa. Posteriormente, se entra en un monitoreo continuo mediante la construcción *mientras (verdadero) hacer*, la cual genera un bucle infinito que resulta apropiado para procesos de supervisión que deben ejecutarse sin interrupción.

Dentro de este ciclo, se ejecuta la adquisición de dato cuando la función *medir_temp(bat_principal)* consulta el sensor térmico asociado al identificador predefinido *bat_principal*. El valor obtenido se somete a una evaluación de umbral, donde la condición *temperatura_actual > 55.0* compara el valor leído contra la constante de seguridad del entorno. Si esta métrica se sobrepasa, se activa la respuesta ante emergencia, encadenando secuencialmente cuatro acciones correctivas: refrigeración, desconexión solar, conmutación de carga industrial y el registro del evento. Finalmente, se aplica la temporización a través de la instrucción *esperar(2000)*, emitiendo una pausa de dos segundos entre ciclos de monitoreo para evitar la saturación del bus de sensores antes de reiniciar la verificación.

Escenario Operativo B: Balance de Carga y Optimización Energética Autónoma

El sistema debe evaluar el estado de carga de los bancos de baterías. Si el nivel es óptimo (superior al 90%) y la generación solar excede la demanda interna, debe vender el

excedente a la red pública. Por el contrario, si la carga desciende por debajo del 20% durante horas de alta demanda nocturna, debe aislar los sectores no esenciales para preservar el suministro en áreas críticas.

Código en Lenguaje L:

```
// Escenario B: Balance de carga y optimización energética autónoma

// Declaración de variables

float carga_actual;

float generacion_solar;

float demanda_interna;

float excedente;

int hora_actual;

// Inicialización del sistema

iniciar_sistema();

// Lectura de variables de estado del sistema

carga_actual = nivel_bateria(bat_principal);

generacion_solar = leer_generacion(panel_solar_array);

demanda_interna = leer_demanda(sensor_principal);

hora_actual = obtener_hora();

// Bloque de decisión para gestión de excedentes (batería óptima)

si (carga_actual > 90.0) entonces {

    si (generacion_solar > demanda_interna) entonces {

        // Cálculo del excedente disponible para inyección a red

        excedente = generacion_solar - demanda_interna;

        // Ejecución de la venta de energía a la red pública
```

```

vender_energia(excedente);

// Registro operativo del evento comercial
// El operador + entre cadena y número realiza conversión
automática a string
    escribir_log("Excedente vendido: " + excedente + " kW");
}

}

// Bloque de decisión para racionamiento autónomo (batería crítica)
si (carga_actual < 20.0) entonces {

    // Verificación de horario nocturno (alta demanda y escasa
generación solar)
    si (hora_actual >= 20 || hora_actual <= 6) entonces {

        // Desconexión de sectores no prioritarios para la operación
aislar_sector(area_no_critica);
aislar_sector(area_logistica);

        // Apagado de cargas secundarias (iluminación exterior)
        // Nota: "apagar" es un comando válido según la especificación
de activar_rele
        activar_rele(iluminacion_exterior, "apagar");

        // Registro de la activación del modo crítico
        escribir_log("MODULO CRITICO: Carga por debajo del 20% en horario
nocturno");
        escribir_log("Sectores no esenciales aislados para preservar
suministro critico");
    }
}

```

}

El bloque de optimización comienza con la adquisición integral de estado, donde cuatro funciones de sensado capturan el porcentaje de carga, la generación fotovoltaica, la demanda eléctrica y la hora del día. Con estos datos, la lógica de optimización (excedentes) evalúa mediante una condición anidada si la batería está llena (> 90.0) y la producción supera el consumo; de cumplirse, el cálculo del excedente determina los kilovatios netos a inyectar a la red pública. Por el contrario, la lógica de racionamiento autónomo detecta si la carga cae a niveles críticos (< 20.0) durante el horario nocturno ($20:00$ a $06:00$), momento de nula generación solar.

Ante esto, el aislamiento selectivo desconecta áreas no esenciales invocando *aislar_sector()* y apaga la iluminación exterior mediante *activar_rele* con el comando "apagar". Todo el proceso mantiene una traza de auditoría con llamadas a *escribir_log()* que registran las decisiones automáticas para facilitar la supervisión humana.

Conclusiones

La culminación de esta investigación formal permite establecer una síntesis analítica que vincula la teoría gramatical pura con las observaciones prácticas de rendimiento y abstracción sintáctica. Los resultados del benchmark evidencian que la teoría de compiladores impacta directamente en la gestión de recursos. Los entornos con compilación nativa basada en LLVM (Zig y Rust) demuestran un control riguroso de la memoria al mantener el consumo pico por debajo de 1.5 MB, eliminando recolectores de basura ocultos. Sin embargo, los tiempos de ejecución observados exponen cómo los motores con tipado dinámico e interpretación (Python y la máquina virtual V8 de JavaScript) implementan optimizaciones internas en tiempo de ejecución que aceleran drásticamente el procesamiento iterativo de colecciones de datos lineales de bajo volumen.

Las diferencias en el análisis sintáctico revelan que una gramática compacta altera la jerarquía del Árbol de Sintaxis Abstracta (AST). El enfoque de iteradores limpios en Rust y Python disminuye los nodos hijos en el AST en comparación con el esquema estándar de tres nodos condicionales de JavaScript, mientras que Zig impone un modelo restrictivo de continuación mediante bucles while que incrementa la seguridad en sistemas. Para concluir, el diseño abstracto del Lenguaje L demuestra que los lenguajes de propósito general pueden ser inadecuados para entornos críticos industrializados donde operan técnicos sin formación en computación avanzada. La formalización de una gramática mediante la notación de Backus-Naur (BNF) permitió restringir el alcance global del software. Esto facilitó el desarrollo de primitivas semánticas en español y reglas de asociatividad a la izquierda capaces de resolver escenarios de alta criticidad, como el aislamiento preventivo de sectores industriales ante fugas térmicas o la inyección automatizada de excedentes energéticos en la microred ECO-GRID sin comprometer la estabilidad del sistema físico simulado.

Referencias bibliográficas

- Chakray. (4 de diciembre de 2018). Lenguajes de programación: tipos, características y diferencias. <https://chakray.com/es/lenguajes-programacion-tipos-caracteristicas/>
- López, F. y Montenegro, M. (2022). Análisis estático de tipos para lenguajes de tipado dinámico [Archivo PDF]. <https://docta.ucm.es/rest/api/core/bitstreams/3e466187-32a2-4ce6-8948-4a4c73a17523/content>
- Montenegro, C. E., Cueva, J. M., Sanjuán, Ó. y Gaona, P. A. (2010). Desarrollo de un lenguaje de dominio específico para sistemas de gestión de aprendizaje y su herramienta de implementación “KiwiDSM” mediante ingeniería dirigida por modelos [Archivo PDF]. <https://dialnet.unirioja.es/descarga/articulo/3869349.pdf>
- Zea, A. (2023). Compiladores de lenguaje específico de dominio (DSL) [Archivo PDF]. https://www.researchgate.net/publication/370655219_COMPILADORES_DE LENGUAJE_ESPECIFICO_DE_DOMINIO_DSL